About the Course Software Testing & Verification

Course Software Testing & Verification

2025/26

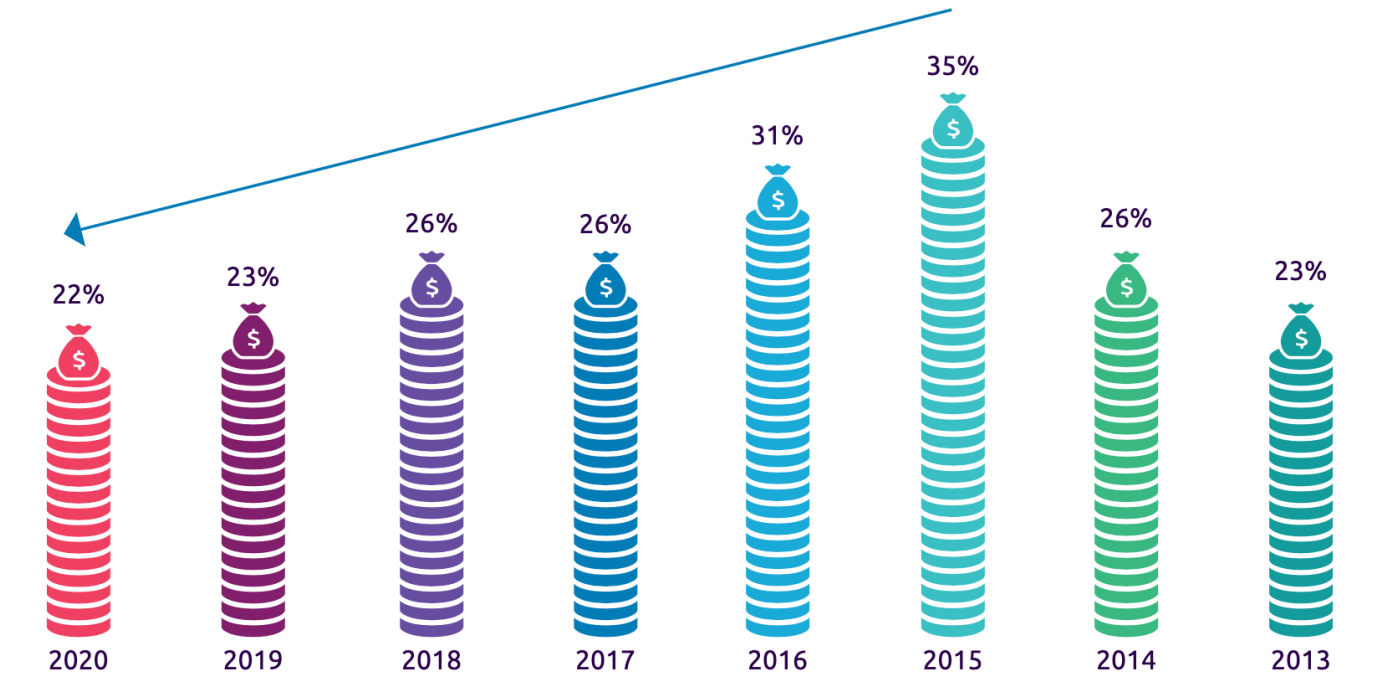
Wishnu Prasetya & Paige North

Why do we care?

1. Because we want to deliver quality products!
2. Because poor quality, including software bugs, may have real consequences

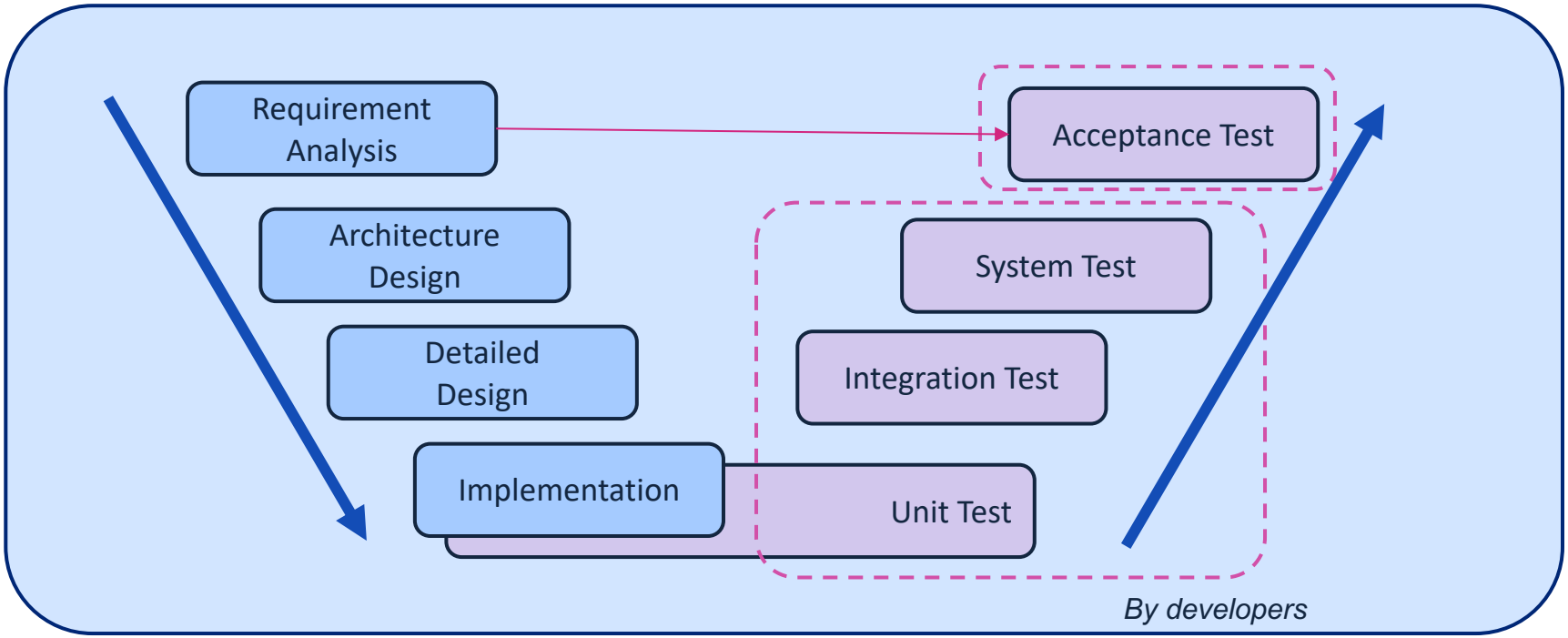
- e.g. some errors in the software of UK Inland Revenue is rumored to cause tax-credit over-payment of 3.45 billions USD. (Charette, *Why Software Fails*. IEEE Spectrum, 2005)
- Uber self-driving accident 2018

Invested effort in quality assurance (QA)



World Quality Report, 2020/21. 1750 mostly CIOs, 32 countries

The project management aspect of quality assurance is non-trivial



A typical testing project approach called “V-model”

In this course we will focus on the technical foundation of software verification

- How to specify what constitutes “correct” behavior?
- How to verify the correctness of a program?
- What constitute good tests ? When have we tested enough?

- Can we automate these steps?

(top level) Learning Goals

- **Know** a selected set of basic concepts, theories, techniques, and technologies of *Software Testing* and *Software Verification*
  They represent two complementary approaches towards software correctness : *pragmatism* vs *completeness*.

- **Able to relate** these theories and techniques to real problems.

Not in scope

- Project management aspects of quality assurance (QA) in a large project
  - covered in Software Project (bachelor).

- Automated verification algorithms
  - covered in the Program Semantic & Verification course (master).
  - see also the “materials”-tab in the website: automated testing, theorem provers, proof assistants, CBMC (bounded model checker for C/C++).

Pre-requisite

- C#
- Working with IDE
- The “software verification” part will go into the mathematical foundation of verification. You will need background in:
  - Set theory
  - Predicate logic

Sites & Materials

- Website is for now hosted in github:
  https://wooshrow.github.io/b3stv/
  https://github.com/wooshrow/b3stv
- Books:
  - Paul Ammann and Jeff Offutt, Introduction to Software Testing, **1st edition**, Cambridge University Press, Cambridge, UK, ISBN 0-52188-038-1, 2008. 2nd Ed is also ok, if you can't get 1st anymore.
  - Lecture Notes (can be obtained from the website), for the program verification part.

Project & assignment

- Software testing Quizz (3x): you can redo them as many times as you want, BUT each has a closing date.
- Writing formal specifications (3x): via MS-Team.
- Testing **Project**
- Proving program correctness **assignment** (3x)

Grading

- In total **12 components**: quizzes (3), formal specifications (3), project, proof assignments (3), 2x exams.
- **Criteria to pass the course**:
  1. You do all exams and projects.
  2. The average of your exams should be ≥ 5.0.
  3. Your score (see below) should be ≥ 6
- *score* = ***if*** 5.0 ≤ *raw* ≤ 6.0 ***then*** *raw rounded to the closest int*
  else *raw rounded to the closest 0.1*
- *raw* = 0.025 * formal specifications
  + 0.225 * proof assignments
  + 0.25 * testing project
  + 0.25 * exam-1 + 0,25 * exam-2

  Your *final* score = "score" (above), except if you didn't meet criteria 1 or 2 above; then your final score would either NVD or AANV.
- Resit: only if you fail and (4.0 ≤ *raw* or *final*=AANV)

Use of AI (ChatGPT etc)

- is NOT allowed! This includes for writing the code for the STV project.
- Exception: you can use them to help you writing tests. Keep in mind that you own the end responsibility, so do check what AI recommends you.

Your Rider IDE has support for copilot AI.

Mark the code that is generated by AI. And specifically add a statement that you have checked the generated code yourself (next slide).

Mark and inspect your AI-generated code

```
/// <summary>
/// test that monsters have the correct initial hp
/// </summary>
/// @ai-generated Inspected by: <your github-id>
monster_test_hp() {
   Monster m = new Monster("id") ;
   assertEqual(10, m.hp()) ;
}
```

Marking AI-generated content is part of good practice and is also demanded by AI transparency Law/s (e.g. the upcoming EU AI Act, 2026).

Software

- Jetbrains Raider IDE, you can get free education license. Works on Windows and Mac, supposedly also on Linux.
- Or Visual Studio **Enterprise** edition**. Not free.**
- You can use Github or uu-Gitlab to host your project. Please make your git-repository private!

Load

- We have a pretty dense programme. Expect to commit at least 16hrs/week.
- Suggested plan:
  - Lectures + lab/wekcollege sessions: 8h/w
  - Self-study the theory: 4h/w
  - Sprints for your Testing Project: 4h/w

Running the course

- Overall week-to-week plan: see website
- Brightspace, for
  - **Testing Quizzes**
  - **Submitting project**
- MS-Team B3STV, for:
  - **Submitting formalization-assignments and Proof-assignments.**
  - **Channels where you can pose questions.**
- Lectures and werkcolleges are physical (on-site) every Tuesday and Thursday. You have the TAs during the werkcolleges.
- I will be monitoring Team-channels outside werkcollege hours.

Team

- Wishnu Prasetya (lectures, werkcollege, Team)
- Paige North (lectures)
- Lana Idema (TA, werkcollege)
- Justin Timmer (TA, werkcollege)
- Tammo Steffens (TA, werkcollege)